

On the Emergence of Thinking in LLMs II: Thinking Big on Small Budgets

Anastasios Giotitsas, Ilias Stylianos Iliadis, Dimitrios Livanas, Stavros Maniatogiannis
NTUA, students (TEAM 05)

Ioannis Kalogiannidis
NTUA, supervisor

Abstract—Recent work has shown that reinforcement learning can induce reasoning behaviors in large language models without relying on expensive process reward models. In this work, we recreate the experimental results of the paper *On the Emergence of Thinking in LLMs I* [1] and investigate the reproducibility of its main findings. Beyond reproducing the original RLSP framework, we extend the study in several directions. First, we implement the dynamic- α exploration schedule proposed but not evaluated in the original work. Second, we evaluate alternative exploration reward functions to better understand the sensitivity of RLSP to reward design. Finally, we investigate Group Relative Policy Optimization (GRPO) as a replacement for PPO within RLSP and compare its behavior with the original optimisation algorithm, including an ablation without the exploration reward. Overall, our results largely confirm the emergence of reasoning behaviors reported in the original paper while providing additional insights into optimisation strategies and reward shaping for reinforcement learning with large language models.

The implementation used throughout this project is available in our [repository](#)

I. INTRODUCTION

A. Large Reasoning Model (LRM)

An LRM is a class of large language model that can perform reasoning at inference time. Unlike a traditional LLM, which directly emits the next token, an LRM is willing to spend extra time and compute during inference in order to produce a substantially higher-quality output. This matches a general definition of “thinking” applicable to any reasoning system, artificial or biological: take more deliberation when the problem is hard.

B. Next-token prediction (supervised learning) is insufficient

- **Pattern-recognition ceiling.** Next-token prediction learns to reproduce statistical patterns of text — it behaves, in the limit, as a “stochastic parrot.” It is excellent at echoing the form of solutions it has seen.
- **The nature of reasoning.** In genuinely hard reasoning (e.g. competition mathematics) we do not want pattern reproduction; we want active manipulation of ideas specific to the problem at hand. The required reasoning path may not even exist in the training data — one cannot reproduce the “pattern” of a proof for a genuinely new proof.
- **Reasoning as guided search.** It is more productive to view reasoning not as prediction but as exploration —

a guided search over the space of ideas. Each transition from one idea to the next is a step that, chained together, forms a Chain of Thought (CoT).

- **Why RL is needed.** We usually do not know in advance whether an individual intermediate step is correct (there is no per-step supervised signal), but we can check whether the final answer is correct. This is exactly the regime where Reinforcement Learning fits: let the model explore on its own and reward it from the outcome

C. From a learned Process Reward Model (PRM) to a fixed-function reward (RLSP)

Earlier attempts to inject reasoning used two separate models:

- 1) **A Policy Model** — the LLM that generates the reasoning steps
- 2) **A Process Reward Model (PRM)** — a separate critic that scores the correctness of each intermediate step.

The drawback is that the PRM is itself trained by supervised learning (as a step-level classifier) on large amounts of human- or strong-model-labelled CoT data. This demands an enormous corpus of specialised annotations and is therefore not scalable.

RLSP removes the need for a learned reward model. Instead of a neural critic it uses a fixed mathematical function for the reward, which combines:

- an **Outcome Verifier** (target-found reward): an objective, binary signal (0/1) that checks only whether the final answer is correct; and
- an **Exploration Reward**: a reward independent of correctness, based e.g. on response length (number of tokens), that encourages the model to produce more intermediate steps and to explore the space of ideas.

Effectively this leaves a single trainable model — the Policy — since the reward is computed automatically by the function. The policy is optionally preceded by an SFT stage (plain next-token prediction on long reasoning traces) so the model first absorbs, to a small degree, the structure and “patterns” of proofs — how to backtrack, how to verify — before self-improvement via RL begins.

D. This report

We ask whether RLSP’s qualitative claims — accuracy gains and, more interestingly, the spontaneous emergence of

reasoning behaviors — survive a roughly 20×–30× reduction in model size (32B → 1.5B, 8B → 1B). Reproducing at small scale is itself informative: if the effect is real and largely about the training signal rather than raw capacity, even a weak signal should bend the model in the right direction. We then propose and run four extensions:

Summary of contributions:

- 1) A **small-scale reproduction** of RLSP on Qwen2.5-1.5B (PPO) and Llama-3.2-1B (GRPO), with qualitative confirmation of emergent behaviors.
- 2) A **dynamic- α** variant that implements the decaying- α schedule the original paper suggests but does not execute.
- 3) Two **alternative exploration rewards** $\left(-\frac{C}{\sqrt{|o|}}, e^{-\sqrt{|o|}}\right)$ that map out RLSP’s sensitivity to reward shape.
- 4) A study of **GRPO as a possible replacement for PPO** inside RLSP, including an ablation that removes the exploration reward.

II. THEORETICAL BACKGROUND: PPO AND GRPO

A. Proximal Policy Optimization (PPO)

PPO was introduced by Schulman et al. [2] to retain the data efficiency and reliable performance of Trust Region Policy Optimization (TRPO) while relying only on first-order optimization methods. In practice, PPO is simpler to implement, more general, and has better sample complexity.

More specifically, PPO is an actor–critic algorithm consisting of i) the current policy model, ii) the old policy model and iii) the reference model, where the policy model is trained jointly with a separate value function, the critic.

The PPO algorithm is shown in Algorithm 1:

Algorithm 1 PPO (Actor–Critic Style)

```

1: for each iteration do
2:   for each actor = 1, ..., N do
3:     Run policy  $\pi_{\theta_{\text{old}}}$  in the environment for  $T$ 
       timesteps
4:     Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
5:   end for
6:   Optimize the surrogate objective  $L(\theta)$  using  $K$  epochs
       and minibatches of size  $M \leq NT$ 
7:    $\theta_{\text{old}} \leftarrow \theta$ 
8: end for
```

In our implementation we follow the PPO algorithm as described in Ye et al. [3], where the model response can be decomposed into tokens as $o = (o_1, o_2, \dots, o_T)$, with $o_T = \text{EOS}$ being the end-of-string token. The aim is to maximize the objective:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (1)$$

where $\rho_t(\theta) = \frac{\pi_\theta(o_t | q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})}$ and $\hat{A}_t$ is the GAE estimate for the advantage function.

The per-token reward used in the advantage function is:

$$r_t = R(q, o) \mathbf{1}(o_t = \text{EOS}) - \beta \log \left(\frac{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})}{\pi_{\text{SFT}}(o_t | q, o_{<t})} \right) \quad (2)$$

The KL term is used in the reward function to prevent drifting too far from the base SFT model $\pi_{\theta_{\text{SFT}}}$.

B. Group Relative Policy Optimization (GRPO)

GRPO is introduced by Shao et al. [4] as a more memory-efficient alternative to PPO by eliminating the need for a separate value function, the critic.

More specifically, GRPO is a group-relative algorithm consisting of i) the current policy model, ii) the old policy model and the iii) the reference model, where the policy model is updated so that it maximizes the GRPO objective.

The GRPO algorithm is shown in Algorithm 2.

Algorithm 2 Iterative Group Relative Policy Optimization

Require: Initial policy $\pi_{\theta_{\text{init}}}$, reward model r_φ , dataset $\mathcal{D}$, hyperparameters ϵ, β, μ

```

1:  $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for  $i = 1, \dots, I$  do
3:    $\pi_{\text{ref}} \leftarrow \pi_\theta$ 
4:   for  $j = 1, \dots, M$  do
5:     Sample batch  $\mathcal{D}_b \subset \mathcal{D}$ 
6:      $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ 
7:     Sample  $G$  outputs  $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$  for each
        $q \in \mathcal{D}_b$ 
8:     Compute rewards  $\{r_i\}_{i=1}^G$ 
9:     Compute group-relative advantages  $\hat{A}_{i,t}$ 
10:    for  $k = 1, \dots, \mu$  do
11:      Update  $\pi_\theta$  by maximizing Eq. (21)
12:    end for
13:    Update  $r_\varphi$  using replay
14:  end for
15: end for
16: return  $\pi_\theta$ 
```

In our implementation however we deviate from the standard GRPO algorithm by introducing a forward KL term, as described in Yao et al. [5] and by omitting the $\frac{1}{|o_i|}$ from the loss function, as proposed in Liu et al. [6]. Furthermore, we use a GRPO variant, by removing PPO-style constraints. This variant is referred as RGRA in Carrino et al. [7].

The loss function we propose is:

$$L_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \left(\log(\pi_\theta(o_{i,t} | q, o_{i,<t})) A_{i,t} - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right). \quad (3)$$

where the advantage is computed by $A_i = \frac{r_i - \mu_{\text{group}}}{\sigma_{\text{group}} + 10^{-8}}$. The term 10^{-8} is used to prevent division with zero and r_i refers to the reward function. The KL penalty we use is:

$$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \sum_{a \in V} \pi_{\theta}(a \mid q, o_{i, < t}) \times \log \left(\frac{\pi_{\theta}(a \mid q, o_{i, < t})}{\pi_{\text{ref}}(a \mid q, o_{i, < t})} \right). \quad (4)$$

An important difference between GRPO and PPO is that in the GRPO algorithm the KL-divergence penalty is not added to the reward, but it is added directly in the optimization objective, simplifying the computation of the advantages.

III. PAPER'S RESULTS

The paper proposes RLSP, a post-training framework in the spirit of RLHF, with three steps:

- **Step 1** — (Optional) **SFT** on demonstrations of the thinking process (human traces, DFS over MCTS, or synthetic long-CoT). The authors use a filtered QwQ-LongCoT dataset; helpful especially for small models.
- **Step 2** — **Exploration reward** $R_{ex}(q, o)$, independent of correctness, rewarding meta-properties of the reasoning trajectory (effort, length, diversity). This rewards the process itself, in contrast to process reward modelling (PRM).
- **Step 3** — **RL with an outcome verifier (PPO)** using a binary correctness signal (final-answer check in math, unit tests in code).

The combined reward is:

$$R(q, o) = a \cdot \mathbb{1}[Ver(q, o) = \text{True}] + (1 - a) \cdot R_{ex}(q, o) \quad (5)$$

and the per-token PPO reward carries a KL penalty to the reference (SFT) policy:

$$r_t = R(q, o) \cdot \mathbb{1}[o_t = EOS] - \beta \log \frac{\pi_{\theta_{old}}(o_t \mid q, o_{< t})}{\pi_{\theta_{SFT}}(o_t \mid q, o_{< t})} \quad (6)$$

The simplest exploration reward is length-based, $R_{ex}(q, o) = -\frac{C}{|o|}$: longer responses (which spell out intermediate steps) receive a higher reward. The motivation is theoretical — more CoT steps provably increase the computational power of transformers [8], [9], so length is necessary (not sufficient) for harder reasoning.

A. Original Paper Results

1) *Llama Models*: Starting from Llama-3.1-8B-Instruct as the base model on which they perform no SFT, they do a) PPO training on the training dataset of MATH with no exploration reward of any kind; that is $a = 1$. b) PPO training on the training dataset of MATH with the creativity reward proportional to length: $R_{ex}(q, o) \propto -\frac{1}{|o|}$. Their results are shown in Fig. 1.

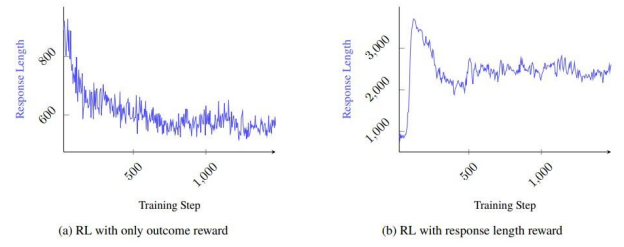


Fig. 1. Response Length as a function of Training Step for Llama-3.1-8B Instruct, with and without length reward

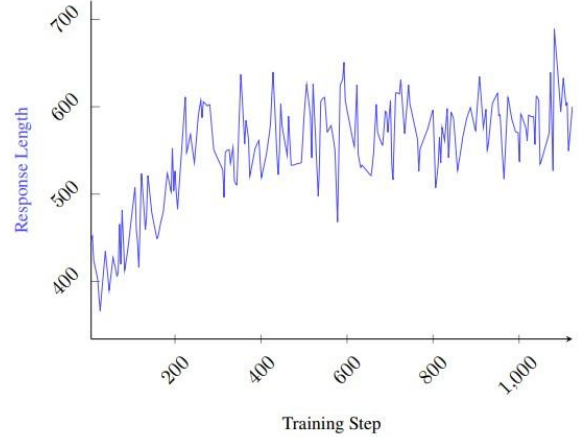


Fig. 2. Response Length as a function of Training Step for Qwen2.5-7B Instruct

2) *Qwen Models*: They also carried out a similar experiment with Qwen2.5-7B-Instruct model in math domain. Their results are shown in Fig. 2.

They next ask if this behavior is specific to the math domain or the Qwen2.5-7B-Instruct model would also exhibit this behavior in other reasoning domains such as coding. Their experiments in the coding domain are shown in Fig. 3.

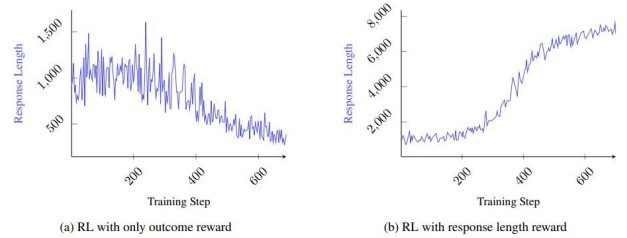


Fig. 3. As shown in Fig. 1, response length explodes after a number of training steps

3) Quantitative Results:

The summary of the scores of the various approaches the original researchers presented is shown in Table. I.

The most emphasised contribution is behavioral: even with the trivial length reward and no SFT, models across fami-

TABLE I
PERFORMANCE OF MODELS WITH AND WITHOUT ER-RLSP ON
MATHEMATICAL REASONING BENCHMARKS.

Model	Setting	Benchmark	Score
Llama-3.1-8B	Base	MATH-500	50.6%
Llama-3.1-8B	SFT	MATH-500	70.2%
Llama-3.1-8B	Base + SFT + ER-RLSP	MATH-500	73.4%
Qwen2.5-32B	Base	AIME 2024	13.3%
Qwen2.5-32B	Base + ER-RLSP	AIME 2024	23.3%
Qwen2.5-32B	Base	MATH-500	81.6%
Qwen2.5-32B	Base + ER-RLSP	MATH-500	83.2%

lies/sizes/domains begin to backtrack, self-verify, self-correct, and explore alternatives — behaviors present but rare in pretraining data and largely absent from vanilla CoT. Ablations show that pure outcome-only RL induces search behavior only sometimes (e.g. Qwen2.5-7B in math but not code), whereas RLSP induces it consistently.

IV. RECREATION

A. Models

- **Qwen:** Qwen2.5-1.5B-Instruct (vs Qwen2.5-32B in the paper).
- **Llama:** Llama-3.2-1B-Instruct (vs Llama-3.1-8B in the paper).

The $\sim 20\text{--}30\times$ downscale is imperative due to much lower computational resources and therefore the central caveat for all comparisons is: absolute accuracies are not comparable to the paper; only directional effects are.

B. Dataset

- **SFT (Llama):** The paper filters QwQ-LongCoT to NuminaMath problems, decontaminates against ~ 25 test sets, ending with 88k samples, we used only 2k of them.
- **RL (Qwen / Llama):** training on (problem, answer) pairs from MATH and AIME (1983–2003 for the AIME variant), with already-solved problems excluded
- **Evaluation:** accuracy on MATH-500

C. Training Procedure

Following the paper as closely as the computational resources allowed:

- **Reward**

$$R(q, o) = a \cdot \mathbb{1}[Ver(q, o) = \text{True}] + (1 - a) \cdot R_{ex}(q, o),$$
with $a = 0.8$ (as in the paper) and length reward $R_{ex} = -\frac{C}{|o|}$, with $C = 1000$ (as in the paper).
- **PPO Parameters:**
 - GAE $\lambda = 0.95$
 - $\gamma = 1$
 - clip $\epsilon = 0.2$
 - For the **KL coefficient**, we used a value of 0.01, whereas the original paper sets it to 0.05. We opted for a smaller KL coefficient due to practical computational constraints, specifically the limited runtime available on our hardware. This allowed the model to tolerate greater drift from the original distribution,

prioritizing the generation of longer sequences within the available computation time.

- **Actor Learning Rate:** $2e-7$ (as in the paper)
- **Critic Learning Rate:** paper $2e-6$ (as in the paper)
- **Rollout:** Because of VRAM limitation 32 instead of the paper’s 512
- **Train Batch:** Because of VRAM limitation 8 instead of the paper’s 128
- Because of limited computational resources, we implemented only 300 training steps, compared with the 700 steps used by the researchers in their corresponding model.

• GRPO Parameters:

- **Group Size** $G = 3$
- **KL coeff:** 0.05
- **Learning Rate:** $1e-6$
- Use of **gradient accumulation**

- **Context:** prompts ≤ 2048 , generations ≤ 2000 tokens (4096 in the paper). Observed response lengths sit around 440–510 tokens, consistent with the small model size.
- Use of **LoRA**, modifying only 0.27% of the total trainable parameters of each model.
- **Hardware:** One L4 GPU was used for the PPO training and two T4 GPUs was used for the GRPO training (less computationally expensive).

D. Results

1) Response Length Behavior:

a) **Llama Models:** Starting from Llama-3.2-1B-Instruct as the base model on which we perform no SFT (like in the original paper), we do

- GRPO training on the training dataset of MATH with no exploration reward of any kind; that is $a = 1$.
- GRPO training on the training dataset of MATH with the creativity reward proportional to length: $R_{ex}(q, o) \propto -\frac{1}{|o|}$, like in the original paper.

Our results are shown in Fig. 4, 5, respectively.

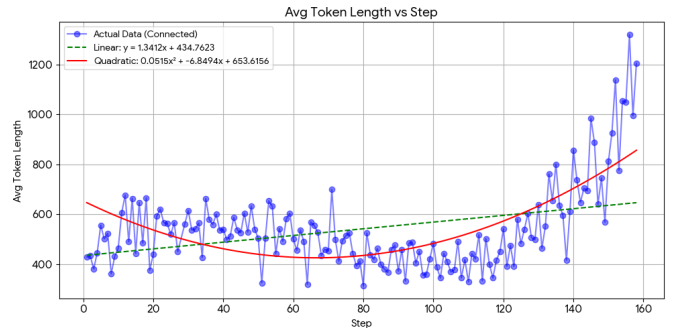


Fig. 4. Step-Response Length function for RLSP training of Llama with exploration reward

These two runs isolate the single most important claim of RLSP at our scale, that the exploration (length) reward, not

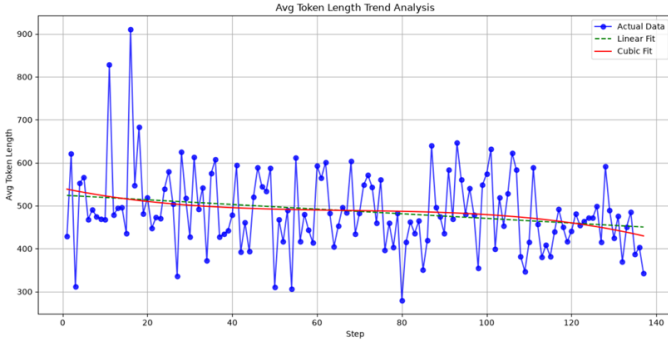


Fig. 5. Step-Response Length function for RLSP training of Llama without exploration reward

the outcome signal, is what drives the model toward longer, more deliberate reasoning.

- **With the exploration reward** (in Fig. 4). We clearly see response length grow over training — it rises substantially from start to finish, with the later steps reaching markedly longer responses than the early ones. This is the length growth that RLSP predicts and the situation in which the emergent behaviors of the paper (backtracking, self-verification) appear.
- **Without the exploration reward** (in Fig. 5). We do not see this behavior. Length stays roughly flat (if anything, slightly lower by the end); the model has no incentive to lengthen its reasoning, so it never takes off.

Takeaway: The exploration reward shapes both training dynamics and the model’s response style: with it, length grows; without it, it does not — reproducing the paper’s ablation at the 1–1.5B scale.

b) Qwen Models: Starting from Qwen2.5-1.5B-Instruct as the base model on which we perform no SFT (like in the original paper), we do PPO training on the training dataset of MATH with the creativity reward proportional to length: $R_{ex} \propto -\frac{1}{|o|}$, like in the original paper. Our results are in Fig. 6, 7.

Reward trends up (training is learning and stable); response length also rises, in the direction RLSP predicts — modestly here because of the small model and $a = 0.8$. The strong length-growth evidence is in the exploration-reward ablation (previous experiment). Be that as is may the beneficial nature of PPO training with this length reward becomes apparent in this model, as shown in Tables II, III

c) Quantitative Results: :

Emergent behaviors (qualitative). Across both models we observe the behaviors the paper highlights in post-RLSP outputs of Qwen2.5-1.5B and Llama-3.2-1B.

E. Discussion of recreation results

- 1) **The direction reproduces.** On Qwen, RLSP improves MATH by +3.6 points (+9% relative). On Llama, the full pipeline (SFT → GRPO) is best at 0.278, and the

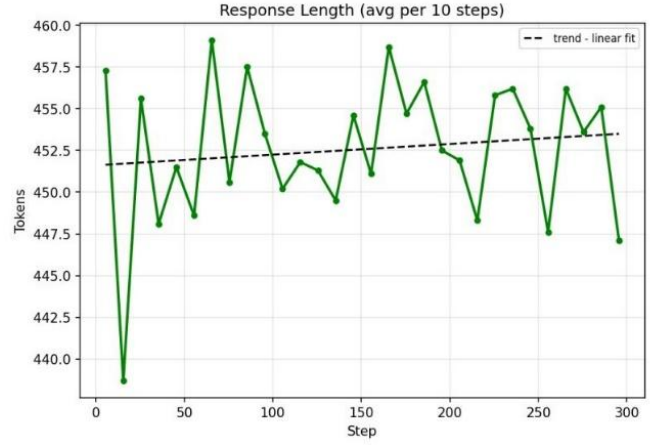


Fig. 6. Step-Response Length function during the RLSP training of Qwen2.5-1.5B-Instruct

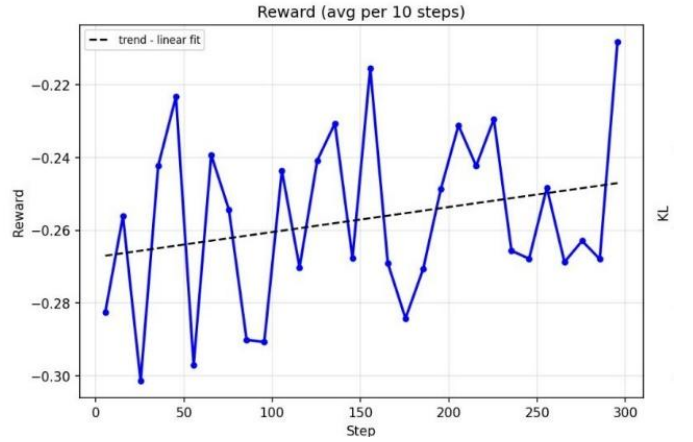


Fig. 7. Step-Reward function during the RLSP training of Qwen2.5-1.5B-Instruct

TABLE II
TRANSFER OF RLSP TRAINING ACROSS DATASETS.

Model	MATH Acc.	Δ vs. Base
Base Qwen2.5-1.5B	0.400	—
RLSP (MATH length reward)	0.436	+0.036 (+9.0%)
RLSP (AIME length reward)	0.404	+0.004

TABLE III
EFFECT OF SFT AND GRPO VARIANTS ON MATH-500 ACCURACY FOR LLAMA-3.2-1B.

Model	Acc.	Δ
Base	0.238	—
Base + GRPO (length exploration, no SFT)	0.258	+0.020
Base + GRPO (no exploration, outcome only)	0.250	+0.012
Base + SFT	0.268	+0.030
Base + SFT + GRPO	0.278	+0.040 (+16.8%)

ordering base (0.238) < outcome-only RL (0.250) < length-RL (0.258) < SFT (0.268) < SFT+RL (0.278)

mirrors the paper’s base $< \text{SFT} < \text{SFT+ER-RLSP}$, with SFT contributing the single largest jump for the small model — exactly the paper’s finding that demonstrations matter most for weak models.

- 2) **The exploration reward is beneficial.** Under GRPO, adding the length-based exploration reward raises the accuracy from 0.250 (outcome-only) to 0.258, small at 1B but positive and in the predicted direction, supporting the claim that rewarding the process helps beyond rewarding only the outcome.
- 3) **AIME $\rightarrow$ MATH transfer is weak at small scale.** The AIME-trained Qwen scores 0.404 on MATH $\approx$ base 0.400. AIME is harder and out-of-distribution for MATH; a 1.5B model extracts little transferable gain. The honest reading is not that RLSP failed but that we evaluated on the wrong benchmark for that run.

V. EXTENSIONS

A. PPO Extensions

1) *Exploration reward — Dynamic a :* The paper fixes a at 0.8, but explicitly states a “needs to be tuned and decayed over time.” We implement that suggestion: instead of a constant trade-off, we schedule a over training. Intuition is curriculum-like — early on, when the verifier signal is sparse (the model rarely solves anything), lean on the dense exploration reward to encourage searching behavior early on. As the model starts solving, shift weight toward correctness so it is paid for being right rather than merely long. This is reminiscent of the exploration VS exploitation dipole in RL training.

Formula for dynamic a :

$$a = a_{\text{start}} + t \cdot (a_{\text{end}} - a_{\text{start}}) \quad (7)$$

$$\text{where } t = \min \left(\frac{\text{current training step}}{\max(m \text{ max training steps}, 1)}, 1 \right)$$

Intuition:

Early in training, when the model rarely solves a problem and the outcome signal is sparse, a lower a leans on the dense exploration reward to push the policy toward longer, more deliberate reasoning. That’s why we encourage exploration at the start (starting in lower values of a). Later, as the model starts producing correct answers more often, raising a shifts the weight back onto correctness, so the model is increasingly rewarded for being right rather than merely for elaborating. That’s why we encourage exploitation as time moves on, by increasing the a coefficient.

Model: Qwen2.5-1.5B-Instruct.

- Dataset: MATH (train), MATH (eval)
- Training: identical PPO setup to subsection. IV-C, with a replaced by the aforementioned formula; everything else (C, lr, GAE, KL) held fixed for a clean comparison.

The results are shown in Table. IV.

Discussion. Dynamic a gives our best Qwen result, 0.444 — beating the fixed- a RLSP run by +0.8 points and the base model by +4.4 points (+11% relative). The gain is consistent with the mechanism the schedule is designed to exploit. In

TABLE IV
COMPARISON OF FIXED AND DYNAMIC a IN RLSP ON MATH ACCURACY.

Model	MATH Acc.
Base Qwen	0.400
RLSP, fixed $\alpha = 0.8$	0.436
RLSP, dynamic a	0.444

practice the improvement over fixed a is modest, as expected at the 1.5B scale and over a short run, but it is consistent and comes at no additional cost — only a change in how the existing two reward components are weighted over time. Overall, dynamic a is the strongest of our PPO extensions and the best-performing Qwen configuration in this study.

2) *Experimenting with the Reward function:* The original paper fixes the exploration reward to the single form $R_{\text{ex}}(q, o) = -\frac{C}{|o|}$ and offers no justification for that particular shape beyond the general principle that longer responses should be rewarded. This leaves an obvious question open: does the exact shape of the exploration reward matter, or is RLSP only sensitive to the fact that the reward rewards length at all? To answer this without an exhaustive sweep, we do not pick two arbitrary alternatives — instead we pick two functions that bracket the paper’s reward from below and from above, so that two runs are enough to probe the entire neighborhood of decay rates around $-\frac{C}{|o|}$, depicted in Fig. 8.

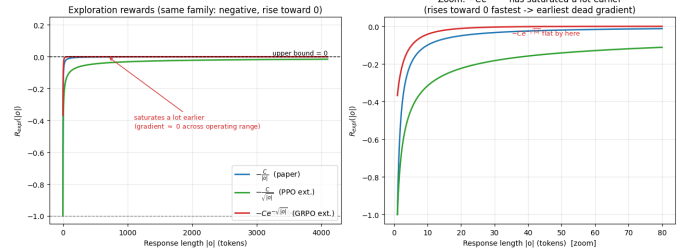


Fig. 8. Plotting of Reward Functions

All three functions belong to the same family: they are negative, bounded, and rise toward the upper bound 0 as $|o|$ grows, so all of them reward longer responses. What separates them is how fast they decay toward 0, and this is exactly what the figure makes visible. Our two choices sit on either side of the paper’s reward:

- Lower bound $\rightarrow -\frac{C}{\sqrt{|o|}}$

This function decays the slowest, so for any given length it stays the most negative (the lowest curve in the figure). Staying further from the asymptote means it retains a non-negligible gradient over a much wider range of lengths, so it provides the strongest sustained incentive to keep elaborating even once responses are already moderately long

- Upper bound $\rightarrow -C \cdot e^{-\sqrt{|o|}}$

This function decays the fastest: it saturates almost immediately and hugs 0 across the entire operating range (the highest curve in the figure). Its gradient effectively fades away after a certain response length, thus providing further length incentive after a certain point. This is beneficial to prevent reward hacking, while still maintaining the response length incentive.

The paper’s own reward, $-\frac{C}{|o|}$, lies between these two. By construction, then, our experiments do not just test two more reward functions — they bound the paper’s design choice on both sides, ranging from a reward whose length incentive is sustained well into long responses (lower bound) to one whose incentive vanishes for longer responses (upper bound). This lets us isolate the role of the reward’s decay rate, and in particular of where along the length axis its gradient is concentrated, as the variable of interest.

a) *Reward Function* $-\frac{C}{\sqrt{|o|}}$: The paper’s $-\frac{C}{|o|}$ has a steep marginal profile: the incentive to add one more token shrinks like $\frac{1}{|o|^2}$, so once responses are moderately long there is little pressure to keep elaborating. We test a gentler decay, $R_{ex} = -\frac{C}{|o|}$, whose marginal reward decays like $\frac{1}{|o|^{1.5}}$ — sustaining the incentive over a wider length range while remaining bounded and monotone.

- Model: Qwen2.5-1.5B-Instruct.
- Dataset: MATH (train/eval, CoT prompt).
- Training: PPO as in subsection. IV-C, only R_{ex} changed.

The results are shown in Table. V.

TABLE V
COMPARISON OF DIFFERENT REWARD FUNCTIONS IN RLSP ON MATH ACCURACY.

Model	MATH Acc.
Base Qwen	0.400
RLSP with $-\frac{C}{ o }$	0.436
RLSP with $-\frac{C}{\sqrt{ o }}$	0.442

The square-root reward (0.442) is marginally above the paper’s form (0.436), and close to dynamic- α (0.444).

Takeaways:

- 1) RLSP is robust to the precise shape of a bounded, monotone exploration reward — the decoupling of exploration from correctness matters more than the exact form
- 2) Gentler decay is at worst harmless and at best slightly beneficial, plausibly because it keeps a useful gradient toward longer CoT for the many mid-length responses a 1.5B model produces.

B. GRPO extensions

1) *GRPO as a different algorithm (the substitution of PPO in itself)*: GRPO is introduced by Shao et al. [4] as a memory-efficient alternative to PPO. While PPO estimates policy advantages using a learned value function (critic), GRPO

removes the critic entirely and instead estimates the relative quality of samples by comparing multiple outputs generated for the same prompt. The reward signal is normalised within each group of sampled responses, producing group-relative advantages that are then used directly to optimise the policy.

This substitution has two practical advantages. First, eliminating the critic substantially reduces memory consumption during training, which is particularly important for large language models where maintaining an additional value network is expensive. Second, avoiding value-function estimation removes a potential source of optimisation instability caused by inaccurate critic predictions. Instead, the optimisation depends only on the relative ranking of candidate responses, making GRPO especially attractive for reinforcement learning from preference or outcome-based rewards.

Conceptually, GRPO can therefore be viewed as a direct replacement for PPO within the RLSP framework: the surrounding reinforcement learning pipeline remains unchanged, while only the policy optimisation algorithm is substituted. One of the main goals of this work is to evaluate whether this replacement preserves the emergence of reasoning behaviors observed with PPO, while potentially offering improved computational efficiency.

2) GRPO with alternative reward form:

- Model: Llama-3.2-1B-Instruct.
- Dataset: MATH (train/eval, Llama prompt).
- Training: GRPO as in subsection. IV-C; we vary (a) the exploration reward shape and (b) the presence of the exploration term in the reward.

The results are shown in Table. VI.

TABLE VI
ABLATION OF EXPLORATION STRATEGIES AND REWARD FORMULATIONS ON LLAMA-3.2-1B.

Model	MATH Acc.	Δ vs. Base
Base Llama-3.2-1B	0.238	—
GRPO (no exploration, outcome-only)	0.250	+0.012
GRPO (length exploration)	0.258	+0.020
GRPO with reward $-C \cdot e^{-\sqrt{ o }}$	0.256	+0.18
Base + SFT	0.268	+0.030
Base + SFT + GRPO	0.278	+0.040

Discussion:

- 1) **Exploration helps.** Removing the exploration reward (0.250) underperforms keeping it (0.258) — the GRPO analogue of the paper’s exploration ablation. Same direction as discussed in subsection. IV-E, now under a different optimiser: a robustness check that passes
- 2) **No significant difference from the change of the Reward function.** Even though $C \cdot e^{-\sqrt{|o|}}$ saturates a lot earlier, the difference in the accuracy is marginal (0.2%).

VI. REPRODUCED EMERGENT BEHAVIORS (EXAMPLES)

Behaviors observed in our own Qwen2.5-1.5B and Llama-3.2-1B RLSP outputs (tags: self_correction, backtracking,

self_verification; most flagged traces are correct=False, consistent with the original paper’s researcher observation, Remark 2 in [1]):

- **Self-Verification:** The model explicitly checks its own reasoning steps to confirm correctness before finalizing an answer. This often involves re-evaluating computations or validating logical consistency.
- **Consideration of alternative possibilities:** The model searches over multiple rationales and explores alternative approaches and recognizes inconsistencies or similarities.
- **Self-Correction:** The model identifies errors in its intermediate steps and revises its solution in the subsequent steps.
- **Backtracking:** Upon identifying errors or dead ends, the model goes back to an earlier step and considers a different reasoning path.

These examples provide strong evidence that RL facilitates the emergence of more systematic and reflective problem-solving behaviors without explicit supervision.

Following the paper’s methodology, we present the emergent behaviors observed in our own RLSP-trained models (Llama-3.2-1B and Qwen2.5-1.5B). Only partial answers are shown, keeping the part reasoning is exhibited, with the steps that exhibit a behavior highlighted in red, as the length of the answers far exceed the intended length of this work, the complete results are accessible [in our repository](#). As in the original work (Remark 2 in [1]), these behaviors emerge without explicit supervision and do not always lead to a correct final answer (every example below reaches a wrong answer).

Self-Correction & Backtracking (Llama-3.2-1B SFT + RLSP, full response on idx:247)

Problem: Let $p(x)$ be a quadratic with integer coefficients having $4 - \sqrt{11}$ as a root. Compute $\frac{p(3)}{p(4)}$.

Answer:

Step 1: By the conjugate root theorem, $4 + \sqrt{11}$ is also a root, so $p(x) = a(x - (4 - \sqrt{11}))(x - (4 + \sqrt{11}))$.

Step 5: Using $p(3) = 0$ gives $p(3) = (3 + \sqrt{11})a$.

Step 6: Since $p(3) = 0$, we get $a = 0$.

However, this contradicts our assumption that a is an integer, so we must have made an error in our previous steps.

Step 7: Revisiting the steps, we realize that we should have used the fact that $p(4) = 0$ since it is a root of $p(x)$.

[...]

Step 19: Revisiting the steps again, we realize that we should have used the fact that $p(4) = 0$ and $p(3) = 0$.

Step 21: Since $p(4) = 0$, we have (8 (out of tokens)

Self-Correction (Qwen2.5-1.5B RLSP on AIME, full response on idx:372)

Problem: In the diagram $AD = BD = CD$ and $\angle BCA = 40^\circ$. Find $\angle BAC$.

Answer:

From the isosceles triangles the model finds $\angle DBC = 70^\circ$ and $\angle BAD = 55^\circ$.

[...]

Hence $\angle BAC = 55^\circ - 70^\circ = -15^\circ$.

However, this result seems incorrect because it doesn’t make sense geometrically.

[...]

Re-evaluating, $\angle BAC = \frac{1}{2}(180^\circ - 40^\circ) = 70^\circ$.

Backtracking (Qwen2.5-1.5B RLSP on AIME, full response on idx:333)

Problem: If $\sqrt[3]{2} = a + \frac{1}{b + \frac{1}{c + \frac{1}{d + \dots}}}$ with a, b, c, d

positive integers, compute b .

Answer:

Writing $x = \frac{1}{b+x}$ and squaring, the model derives $b^2 + 4ab + 4a^2 - 4 = 0$, giving $b = -2a + 2$; with $a = 1$ this yields $b = 0$.

[...]

However, since b must be a positive integer, there seems to be an error in our initial assumption or simplification. Let’s re-evaluate the problem.

Revisiting the steps, we realize that the correct approach should involve solving the cubic equation directly rather than squaring incorrectly.

[...]

Final: $b = 1$.

VII. CONCLUSION

In this work, we successfully recreated the main experimental findings of *On the Emergence of Thinking in LLMs I*, confirming that reinforcement learning with the RLSP framework can encourage the emergence of reasoning behaviors in large language models without requiring process-level supervision. Reproducing the original results provided valuable insight into the training pipeline and demonstrated that the reported behaviors are largely reproducible under comparable experimental settings.

Beyond replication, we extended the original work in several meaningful ways. We implemented and evaluated a dynamic- α exploration schedule, investigated alternative reward formulations, and explored the use of Group Relative Policy

Optimization (GRPO) as a replacement for PPO within the RLSP framework. These experiments provide a broader understanding of how optimisation algorithms and reward design influence the emergence of reasoning, while highlighting the flexibility of RLSP as a reinforcement learning framework.

Although our results generally support the conclusions of the original paper, they also reveal that performance remains sensitive to the choice of optimisation strategy, reward function, and training configuration. This suggests that there is still considerable room for improving both the stability and efficiency of reinforcement learning for reasoning-oriented language models.

Overall, this project provided practical experience in reproducing recent research on reinforcement learning for large language models and evaluating several extensions to the original methodology. Recreating the experiments helped us better understand the RLSP framework, while the additional experiments with dynamic exploration schedules, alternative reward functions, and GRPO offered further insight into the impact of different design choices. The experience of reproducing published work and evaluating several modifications offered a deeper understanding of reinforcement learning techniques for large language models and the practical challenges involved in implementing and analyzing them.

VIII. FUTURE WORK

Although this work successfully recreates the main findings of *On the Emergence of Thinking in LLMs I* and explores several extensions, there remain multiple promising directions for future research.

- **Evaluation across broader domains and datasets.** The RLSP framework should be evaluated on alternative datasets and domains beyond mathematical reasoning. Testing its effectiveness on programming, scientific reasoning, planning, commonsense reasoning, and other challenging benchmarks would help determine whether the observed emergence of reasoning is domain-independent or primarily influenced by the characteristics of mathematics datasets.
- **Integration of richer reward signals.** The reward function could be expanded by incorporating an LLM-as-a-Judge. Instead of rewarding only reasoning length or correctness, an evaluator model could assess multiple dimensions of a response, including logical coherence, creativity, originality, robustness, and out-of-the-box thinking. Such richer feedback signals may encourage more diverse and higher-quality reasoning strategies.
- **Further exploration of optimisation schedules and reward functions.** Additional experiments should investigate alternative dynamic fine-tuning schedules and reward formulations. The dynamic- α schedule explored in this work represents only one possible curriculum strategy; alternative decay schedules, adaptive exploration mechanisms, or reward combinations may achieve a better balance between exploration and exploitation during training.

- **Complete cross-design evaluation of optimisation algorithms and model families.** A comprehensive study combining different optimisation algorithms with different model architectures would help isolate the effects of the optimisation method from those of the underlying language model. In this work, PPO was evaluated on Qwen while GRPO was evaluated on Llama. Evaluating the remaining configurations (GRPO on Qwen and PPO on Llama) would provide a fairer and more complete comparison between PPO and GRPO.

Beyond these extensions, several additional improvements could strengthen the RLSP framework. More extensive hyperparameter optimisation would clarify the sensitivity of the method to training settings. Scaling experiments to larger models and longer training horizons would help determine whether the observed reasoning behaviors continue to improve with increased compute. Finally, analysing intermediate chains of thought and failure cases could provide deeper insight into how reasoning emerges during reinforcement learning and guide the design of more effective reward functions.

REFERENCES

- [1] G. Ye, K. D. Pham, X. Zhang, S. Gopi, B. Peng, B. Li, J. Kulkarni, and H. A. Inan, “On the Emergence of Thinking in LLMs I: Searching for the Right Intuition”, arXiv preprint arXiv:2502.06773, 2025.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, Aug. 2017.
- [3] C. Ye, Z. Yu, Z. Zhang, H. Chen, N. Sadagopan, J. Huang, and T. Zhang, “Beyond Correctness: Harmonizing Process and Outcome Rewards through RL Training,” arXiv:2509.03403, 2025.
- [4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, D. Guo, and D. Yang, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” arXiv:2402.03300, 2024.
- [5] J. Yao, R. Wang, H. Bai, and T. Zhang, “Future-KL Regularized GRPO: Process-Level Credit Assignment from f -Divergence Regularization,” arXiv:2601.10201, 2026.
- [6] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin, “Understanding R1-Zero-Like Training: A Critical Perspective,” arXiv:2503.20783, 2025.
- [7] G. Carrino, A. Sassella, N. Brunello, F. Toschi, and M. J. Carman, “Are Complicated Loss Functions Necessary for Teaching LLMs to Reason?” arXiv:2603.18756, 2026.
- [8] Z. Li, Y. Cao, X. Xu, J. Jiang, X. Liu, Y. S. Teo, S.-W. Lin, and Y. Liu, “LLMs for Relational Reasoning: How Far are We?” in *Proc. 1st Int. Workshop on Large Language Models for Code (LLM4Code)*, pp. 119–126, 2024, doi: 10.1145/3643795.3648387.
- [9] W. Merrill and A. Sabharwal, “The Parallelism Tradeoff: Limitations of Log-Precision Transformers,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 531–545, 2023.